フォーマットファイルアップロードダイアログ UI仕様書

1. 概要

1.1 コンポーネントの目的

フォーマットファイルアップロードダイアログは、選択されたフォーマットフォルダまたはサブフォルダに、単一ファイル・複数ファイル・フォルダ階層を一括でアップロードするためのモーダルダイアログです。ドラッグ&ドロップに対応し、アップロード進行状況をリアルタイムで表示します。

1.2 使用場面

- **表示場所**: フォーマット管理画面のモーダルオーバーレイ
- **トリガー**:「ファイルアップロード」ボタンのクリック
- **主な用途**:
  - XMLテンプレートファイルの一括アップロード
  - MAIMLワークフローファイルのアップロード
  - フォルダ階層を維持したアップロード
  - サブフォルダへのファイル追加

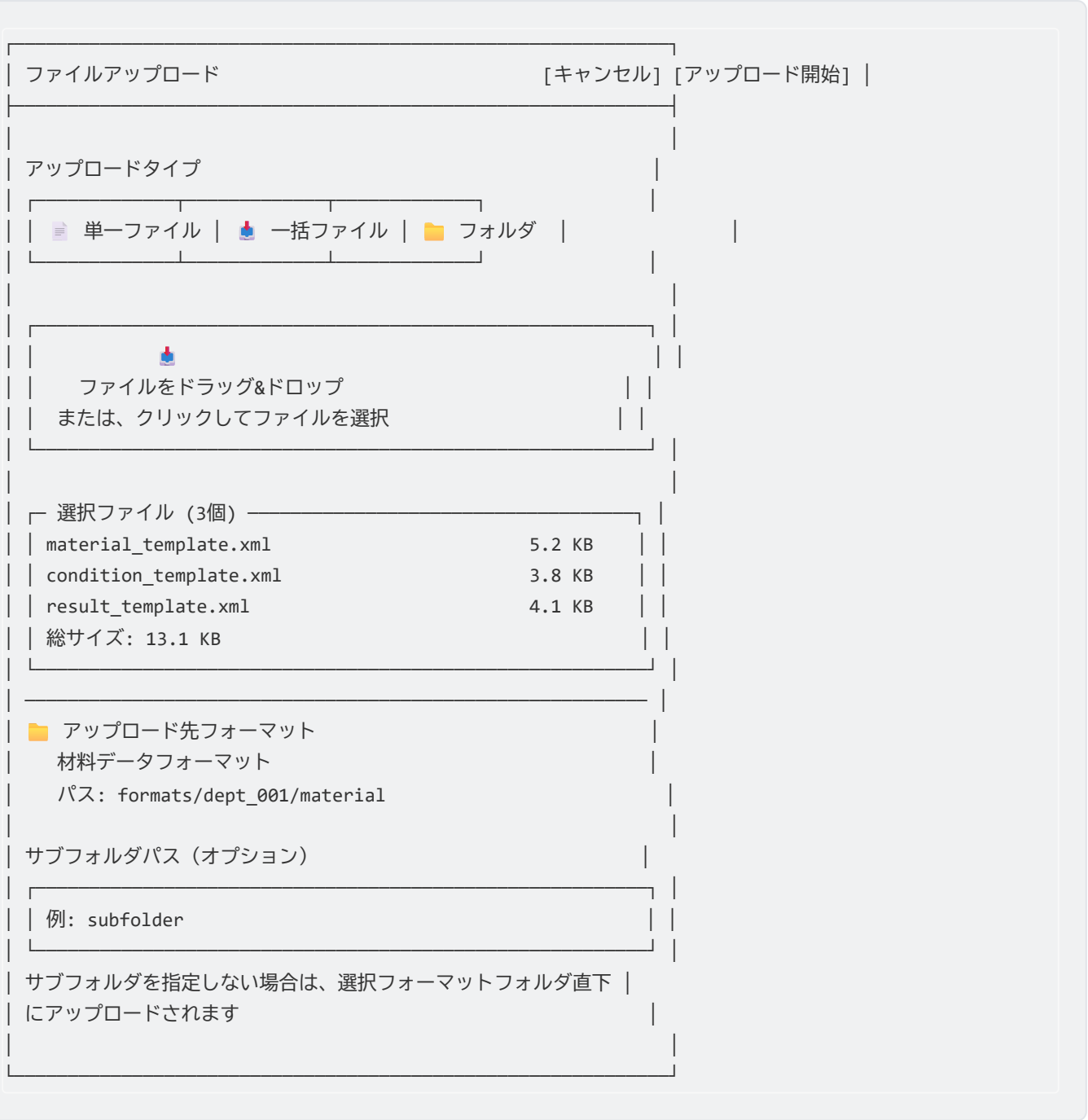
1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: 親コンポーネント（FormatManagement）で認証済み

---

2. 画面構成

2.1 モーダル構造



2.2 主要UI要素

2.2.1 モーダルヘッダー

- **タイトル**:「ファイルアップロード」
- **アクションボタン**:
  - キャンセルボタン

- アップロード開始ボタン（プライマリ）

2.2.2 アップロードタイプセグメント

- **タイプ**: Segmented（ボタングループ）
- **オプション**:
  1. 単一ファイル（📄 FileOutlined）
  2. 一括ファイル（📥 InboxOutlined）
  3. フォルダ（📁 FolderOutlined）

2.2.3 ファイル選択エリア

単一ファイル・一括ファイルモード

- **タイプ**: Dragger（ドラッグ&ドロップエリア）
- **アイコン**: InboxOutlined（青色、#0093CF）
- **メインテキスト**:「ファイルをドラッグ&ドロップ」
- **サブテキスト**:「または、クリックしてファイルを選択」
- **対応形式**: .maiml, .xml, .json, .txt, .csv, .png, .jpg, .jpeg, .gif, .pdf, .tif, .tiff, .bmp, .svg, .webp

フォルダモード

- **タイプ**: Button（フォルダ選択ボタン）
- **アイコン**: FolderOutlined
- **テキスト**:「フォルダを選択」
- **機能**: 非表示のinput[type="file"]をトリガー（webkitdirectory属性）

2.2.4 選択ファイル表示エリア（条件付き）

- **表示条件**: ファイルが選択されている場合
- **背景色**: #f5f5f5
- **内容**:
  - ファイル数
  - ファイル名とサイズのリスト（最大10件表示）

- 11件目以降は「他 N ファイル...」と表示
- 総サイズ（青い背景ボックス）

2.2.5 アップロード先フォーマット表示

- **背景色**: #E6F7FC（水色）
- **ボーダー**: #0093CF（青）
- **内容**:
  - フォーマット名
  - フォーマットパス

2.2.6 サブフォルダパス入力

- **タイプ**: Input
- **プレースホルダー**: 「例: subfolder（空欄の場合は選択フォーマットフォルダ直下）」
- **補足説明**: サブフォルダを指定しない場合の挙動

2.2.7 アップロード進行状況（条件付き）

- **表示条件**: アップロード中
- **タイプ**: Alert（info、icon=LoadingOutlined）
- **内容**:
  - 「N / M ファイル完了」
  - Progress（進行状況バー）

2.2.8 アップロード完了表示（条件付き）

- **表示条件**: アップロード完了後
- **タイプ**: Alert（success、icon=CheckCircleOutlined）
- **内容**: 「N個のファイルが正常にアップロードされました」

2.2.9 エラー表示（条件付き）

- **表示条件**: エラー発生時
- **タイプ**: Alert（error、icon=CloseCircleOutlined、closable）
- **内容**: エラーメッセージ

2.3 インタラクション

2.3.1 モーダル表示

- **操作**:「ファイルアップロード」ボタンをクリック
- **結果**: モーダルが開き、選択されたフォーマット情報が表示される

2.3.2 アップロードタイプの選択

- **操作**: Segmentedボタンをクリック
- **結果**:
  - 選択したモードに切り替わる
  - ファイルリストがクリアされる
  - ファイル選択エリアが切り替わる

2.3.3 ファイル選択（単一・一括モード）

- **操作1**: ファイルをドラッグ&ドロップ
- **操作2**: クリックしてファイル選択ダイアログを開く
- **結果**:
  - 選択ファイルが表示される
  - 単一モードの場合、最後の1ファイルのみが残る

2.3.4 フォルダ選択（フォルダモード）

- **操作**:「フォルダを選択」ボタンをクリック
- **結果**:
  - システムのフォルダ選択ダイアログが開く

- フォルダ内の全ファイルが選択される
- ファイル名にパスが含まれる（webkitRelativePath）

2.3.5 サブフォルダパスの入力

- **操作**: サブフォルダパス入力フィールドに入力
- **結果**: アップロード先パスが動的に更新される

2.3.6 アップロード開始

- **操作**: 「アップロード開始」ボタンをクリック
- **結果**:
  - アップロード処理が開始される
  - 進行状況が表示される
  - 完了後、成功メッセージが表示される

2.3.7 アップロード中のキャンセル

- **操作**: 「キャンセル」ボタンをクリック
- **結果**: 確認ダイアログが表示される

2.3.8 モーダルを閉じる

- **操作**: ×ボタン、またはキャンセルボタンをクリック
- **結果**:
  - アップロード中でない場合: 即座に閉じる
  - アップロード中の場合: 確認ダイアログが表示される

---

3. 機能仕様

3.1 アップロードモード

3.1.1 単一ファイルモード

```
type UploadMode = 'single';
```

- **機能**: 1つのファイルのみをアップロード
- **ファイル選択**: 複数選択しても最後の1ファイルのみが残る
- **用途**: テンプレートファイルの個別アップロード

3.1.2 一括ファイルモード

```
type UploadMode = 'bulk';
```

- **機能**: 複数のファイルを同時にアップロード
- **ファイル選択**: 複数選択可能
- **用途**: 複数のテンプレートファイルの一括アップロード

3.1.3 フォルダモード

```
type UploadMode = 'folder';
```

- **機能**: フォルダ階層を維持したままアップロード
- **ファイル選択**: フォルダ選択ダイアログ
- **用途**: サブフォルダ構造を含むファイル群のアップロード

3.1.4 UI動作

- ユーザー操作: Segmentedボタンをクリック
- 画面変化:

- 選択したモードに切り替わる
- ファイルリストがクリアされる
- ファイル選択UIが切り替わる

3.2 ファイル選択処理

3.2.1 ファイル選択ハンドラ（単一・一括モード）

```
const handleFileChange: UploadProps['onChange'] = useCallback(
  (info: { fileList: UploadFile[] }) => {
    let newFileList = [...info.fileList];

    // 単一ファイルモードの場合は1つのみ
    if (uploadMode === 'single') {
      newFileList = newFileList.slice(-1);
    }

    setFileList(newFileList);
  },
  [uploadMode]
);
```

3.2.2 フォルダ選択ハンドラ（フォルダモード）

```
const handleFolderSelect = useCallback(
  (event: React.ChangeEvent<HTMLInputElement>) => {
    const files = event.target.files;
    if (!files || files.length === 0) return;

    const uploadFiles: UploadFile[] = Array.from(files).map((file, index) => {
      const fileWithPath = file as File & { webkitRelativePath?: string };
      const rcFile = file as RcFile;
      rcFile.uid = `${Date.now()}-${index}`;

      return {
        uid: rcFile.uid,
        name: fileWithPath.webkitRelativePath || file.name,
        status: 'done' as const,
        size: file.size,
        type: file.type,
```

```
        originFileObj: rcFile,
      };
    });

    setFileList(uploadFiles);
  },
  []
);
```

- **webkitRelativePath**: フォルダからの相対パスが取得される
- **例**: `subfolder1/subfolder2/file.xml`

3.2.3 自動アップロード無効化

```
const beforeUpload = useCallback(() => {
  return false; // 自動アップロードを無効化
}, []);
```

- **理由**: アップロード開始ボタンをクリックするまでアップロードしない

3.2.4 UI動作

- ユーザー操作: ファイル選択またはドラッグ&ドロップ
- 画面変化:
  - 選択ファイルリストが表示される
  - 総サイズが計算・表示される

3.3 アップロード先パスの構築

3.3.1 パス構築ロジック

```
const getTargetPath = useCallback(() => {
  if (!baseFolder) {
    return '';
```

```
  }

  // formats/{departmentCd}/{formatFolderName} または formats/{formatFolderName}
  const basePath = baseFolder.departmentCd
    ? `formats/${baseFolder.departmentCd}/${baseFolder.folderName}`
    : `formats/${baseFolder.folderName}`;

  // サブフォルダパスから先頭・末尾のスラッシュを削除
  const cleanSubFolder = subFolderPath.trim().replace(/^\/+|\/+$/g, '');

  // サブフォルダパスはフォーマットフォルダルートからの絶対パスとして扱う
  const finalPath = cleanSubFolder ? `${basePath}/${cleanSubFolder}` : basePath;

  return finalPath;
}, [baseFolder, subFolderPath]);
```

例:

- フォーマットフォルダ直下: `formats/dept_001/material`
- サブフォルダ指定: `formats/dept_001/material/subfolder1`

3.3.2 UI動作

- ユーザー操作: サブフォルダパスを入力
- 画面変化:
  - アップロード先パス表示が動的に更新される

3.4 アップロード実行

3.4.1 バリデーション

```
if (fileList.length === 0) {
  message.warning('ファイルを選択してください');
  return;
}

if (!baseFolder) {
  message.error('フォーマットが選択されていません');
```

```
  return;
}
```

3.4.2 アップロード処理

```
const handleUpload = useCallback(async () => {
  setUploadState({
    isUploading: true,
    progress: 0,
    uploadedCount: 0,
    totalCount: fileList.length,
    error: null,
  });

  try {
    // originFileObjを抽出してFile型に変換
    const files = fileList
      .map(f => {
        const originFile = f.originFileObj;
        if (!originFile) return null;

        // フォルダモードで、かつnameにパスが含まれている場合
        if (uploadMode === 'folder' && f.name.includes('/')) {
          // nameにパスを含む新しいFileオブジェクトを作成
          return new File([originFile], f.name, {
            type: originFile.type,
            lastModified: originFile.lastModified,
          });
        }

        return originFile as File;
      })
      .filter((f): f is File => !!f);

    const targetPath = getTargetPath();

    // API呼び出し
    const response = await uploadFiles({
      uploadMode,
      targetPath,
      files,
      metadata: {
        formatId: baseFolder.id,
      },
      onProgress: (progress) => {
        setUploadState(prev => ({
```

```
          ...prev,
          progress: progress.percentage,
          uploadedCount: progress.uploadedCount,
        }));
      },
    });

    if (response) {
      message.success('アップロードが完了しました');

      if (onUploadComplete) {
        onUploadComplete({
          mode: uploadMode,
          fileCount: fileList.length,
          success: true,
        });
      }

      // 完了後の状態リセット（2秒後）
      setTimeout(() => {
        setFileList([]);
        setSubFolderPath('');
        setUploadState({
          isUploading: false,
          progress: 100,
          uploadedCount: fileList.length,
          totalCount: fileList.length,
          error: null,
        });
      }, 2000);
    }
  } catch (error) {
    const errorMessage = extractErrorMessage(error);
    setUploadState(prev => ({
      ...prev,
      isUploading: false,
      error: errorMessage,
    }));
    message.error(errorMessage);
  }
}, [fileList, uploadMode, baseFolder, getTargetPath, onUploadComplete, message]);
```

3.4.3 プログレスコールバック

```
onProgress: (progress) => {
  setUploadState(prev => ({
```

```
    ...prev,
    progress: progress.percentage,
    uploadedCount: progress.uploadedCount,
  }));
}
```

- **更新頻度**: ファイルごとに更新
- **表示内容**: パーセンテージとアップロード済みファイル数

3.4.4 UI動作

- ユーザー操作:「アップロード開始」ボタンをクリック
- 画面変化:
  - ボタンがローディング状態になる
  - 進行状況が表示される
  - 完了後、成功メッセージが表示される
  - 2秒後に状態がリセットされる

3.5 エラーハンドリング

3.5.1 ファイル未選択エラー

```
if (fileList.length === 0) {
  message.warning('ファイルを選択してください');
  return;
}
```

3.5.2 フォーマット未選択エラー

```
if (!baseFolder) {
  message.error('フォーマットが選択されていません');
  return;
}
```

3.5.3 API通信エラー

```
catch (error) {
  const errorMessage = extractErrorMessage(error);
  setUploadState(prev => ({
    ...prev,
    isUploading: false,
    error: errorMessage,
  }));
  message.error(errorMessage);
}
```

3.5.4 UI動作

- ユーザー操作: なし（エラー発生時に自動表示）
- 画面変化:
  - エラーAlertが表示される
  - Ant Design Messageが画面上部に表示される

3.6 アップロード中のキャンセル

3.6.1 キャンセル確認

```
const handleClose = useCallback(() => {
  if (uploadState.isUploading) {
    Modal.confirm({
      title: 'アップロード中',
      content: 'アップロード処理が進行中です。キャンセルしますか？',
      okText: 'はい',
      cancelText: 'いいえ',
      onOk: onClose,
    });
  } else {
    onClose();
  }
}, [uploadState.isUploading, onClose]);
```

3.6.2 UI動作

- ユーザー操作: アップロード中に「キャンセル」ボタンをクリック
- 画面変化:
  - 確認ダイアログが表示される
  - 「はい」をクリックすると、モーダルが閉じる
  - 「いいえ」をクリックすると、アップロード続行

---

4. データ仕様

4.1 Props定義

```
interface FormatUploadDialogProps {
  open: boolean;                                    // ダイアログの表示/非表示
  onClose: () => void;                              // 閉じるハンドラ
  onUploadComplete?: (result: {                     // アップロード完了ハンドラ
    mode: UploadMode;
    fileCount: number;
    success: boolean;
  }) => void;
  baseFolder: {                                     // ベースフォルダ（選択されたフォーマット）
    id: string;
    name: string;
    folderName: string;
    departmentCd?: string;
  } | null;
  currentFolderPath?: string[];                     // 現在開いているフォルダパス
}
```

4.2 UploadMode型定義

```
type UploadMode = 'single' | 'bulk' | 'folder';
```

4.3 UploadState型定義

```
interface UploadState {
  isUploading: boolean;       // アップロード中フラグ
  progress: number;           // 進行状況（0-100）
  uploadedCount: number;      // アップロード済みファイル数
  totalCount: number;         // 総ファイル数
  error: string | null;       // エラーメッセージ
}
```

4.4 State定義

```
const [uploadMode, setUploadMode] = useState<UploadMode>('single');
const [fileList, setFileList] = useState<UploadFile[]>([]);
const [subFolderPath, setSubFolderPath] = useState<string>('');
const [uploadState, setUploadState] = useState<UploadState>({
  isUploading: false,
  progress: 0,
  uploadedCount: 0,
  totalCount: 0,
  error: null,
});
```

---

5. API連携

5.1 呼び出されるAPI

```
const response = await uploadFiles({
  uploadMode,
  targetPath,
  files,
  metadata: {
    formatId: baseFolder.id,
  },
  onProgress: (progress) => {
```

```
    // プログレス更新
  },
});
```

API:

- **関数**: `uploadFiles`（fileAccessService）
- **パラメータ**:
  - `uploadMode`: アップロードモード
  - `targetPath`: アップロード先パス
  - `files`: Fileオブジェクトの配列
  - `metadata`: メタデータ（formatId）
  - `onProgress`: プログレスコールバック

---

6. エラーハンドリング

6.1 エラーケースと表示方法

エラーケース	検知タイミング	表示方法	表示内容
ファイル未選択	アップロード開始時	Ant Design Message	「ファイルを選択してください」
フォーマット未選択	アップロード開始時	Ant Design Message	「フォーマットが選択されていません」
API通信エラー	API返却時	Alert + Message	エラーメッセージ

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/project/FormatUploadDialog.tsx`

7.2 関連ファイル

- **親コンポーネント**: `frontend/src/components/project/FormatManagement.tsx`
- **サービス**: `frontend/src/services/fileAccessService.ts` (uploadFiles)
- **ユーティリティ**: `frontend/src/utils/errorUtils.ts` (extractErrorMessage)

7.3 依存ライブラリ

- **Ant Design**: Modal, Segmented, Upload (Dragger), Button, Input, Alert, Progress, Space, Typography, Divider, App
- **Ant Design Icons**: UploadOutlined, InboxOutlined, FolderOutlined, FileOutlined, CheckCircleOutlined, CloseCircleOutlined, LoadingOutlined

---

8. UI/UXの考慮事項

8.1 レスポンシブデザイン

- **幅**: 800px（固定）
- **高さ**: 最大70vh、スクロール可能

8.2 アクセシビリティ

- **ドラッグ&ドロップ**: 代替としてクリック選択も可能
- **キーボード操作**: フォーカス管理とEnter/Escキー対応

8.3 視覚的フィードバック

- **進行状況**: リアルタイムでProgress表示
- **完了状態**: 緑色のAlert表示
- **エラー状態**: 赤色のAlert表示

8.4 ユーザーガイダンス

- **補足説明**: サブフォルダパスの説明
- **プレースホルダー**: 入力例を表示

---

9. 変更履歴

日付	変更内容	担当者
2026-03-11	実装ベースで仕様書を作成（Phase 4: フォーマット管理画面）	Cline